In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

In [2]:
```
df = pd.read_csv('kc_house_data.csv')
df.head()
```

Out[2]:

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000
3	2487200875	20141209T000000	604000.0	4	3.00	1960	5000
4	1954400510	20150218T000000	510000.0	3	2.00	1680	8080

5 rows × 21 columns

In [3]:
```
df.shape
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 21613 entries, 0 to 21612
Data columns (total 21 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   id             21613 non-null  int64
 1   date           21613 non-null  object
 2   price          21613 non-null  float64
 3   bedrooms       21613 non-null  int64
 4   bathrooms      21613 non-null  float64
 5   sqft_living    21613 non-null  int64
 6   sqft_lot       21613 non-null  int64
 7   floors         21613 non-null  float64
 8   waterfront     21613 non-null  int64
 9   view           21613 non-null  int64
 10  condition      21613 non-null  int64
 11  grade          21613 non-null  int64
 12  sqft_above     21613 non-null  int64
 13  sqft_basement  21613 non-null  int64
 14  yr_built       21613 non-null  int64
 15  yr_renovated   21613 non-null  int64
 16  zipcode        21613 non-null  int64
 17  lat            21613 non-null  float64
 18  long           21613 non-null  float64
 19  sqft_living15  21613 non-null  int64
 20  sqft_lot15     21613 non-null  int64
dtypes: float64(5), int64(15), object(1)
memory usage: 3.5+ MB
```

In [4]:
```python
df.describe()
```

Out[4]:

	id	price	bedrooms	bathrooms	sqft_living	sqf
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000000	2.161300
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899736	1.510697
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440897	4.142051
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000000	5.200000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000000	5.040000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000000	7.618000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000000	1.068800
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000000	1.651359

In [5]:
```python
df.isnull().sum()
```

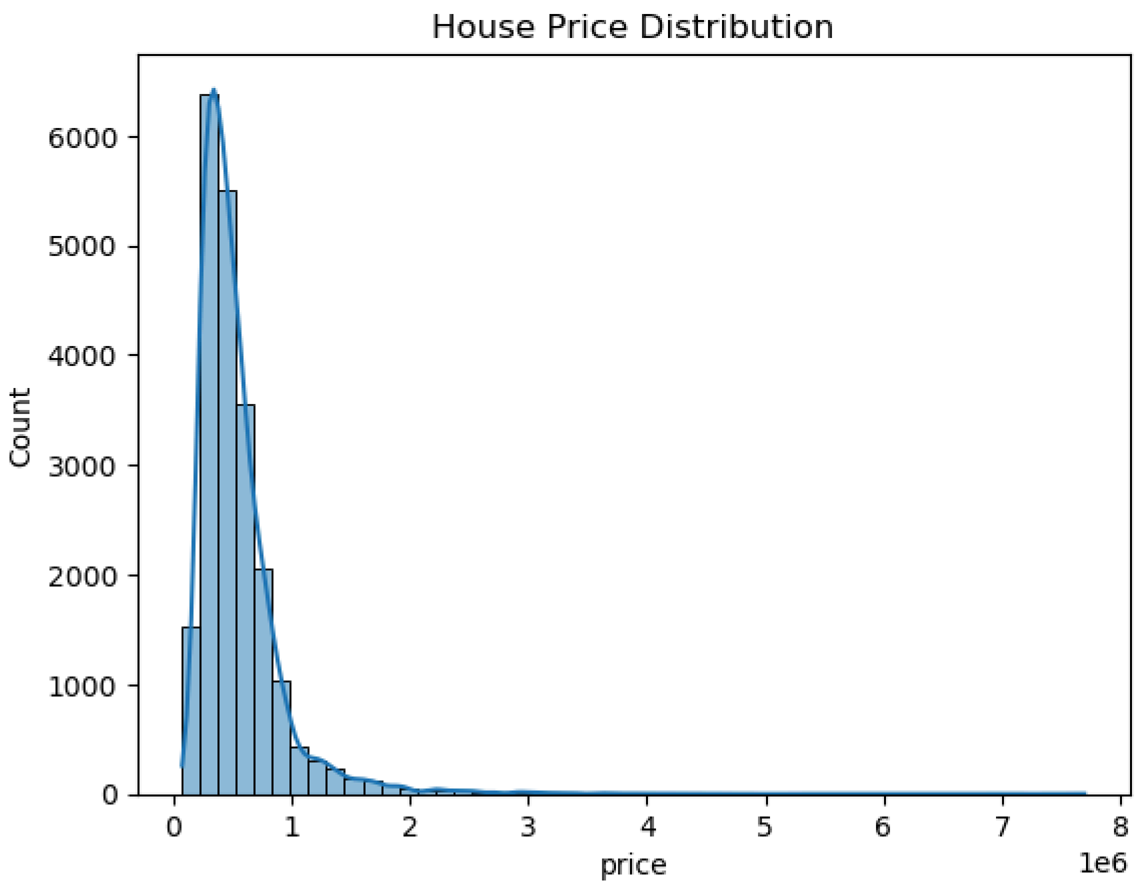
Out[5]:
```
id               0
date             0
price            0
bedrooms         0
bathrooms        0
sqft_living      0
sqft_lot         0
floors           0
waterfront       0
view             0
condition        0
grade            0
sqft_above       0
sqft_basement    0
yr_built         0
yr_renovated     0
zipcode          0
lat              0
long             0
sqft_living15    0
sqft_lot15       0
dtype: int64
```

In [6]:
```python
df.drop(['id', 'date'], axis=1, inplace=True)
```

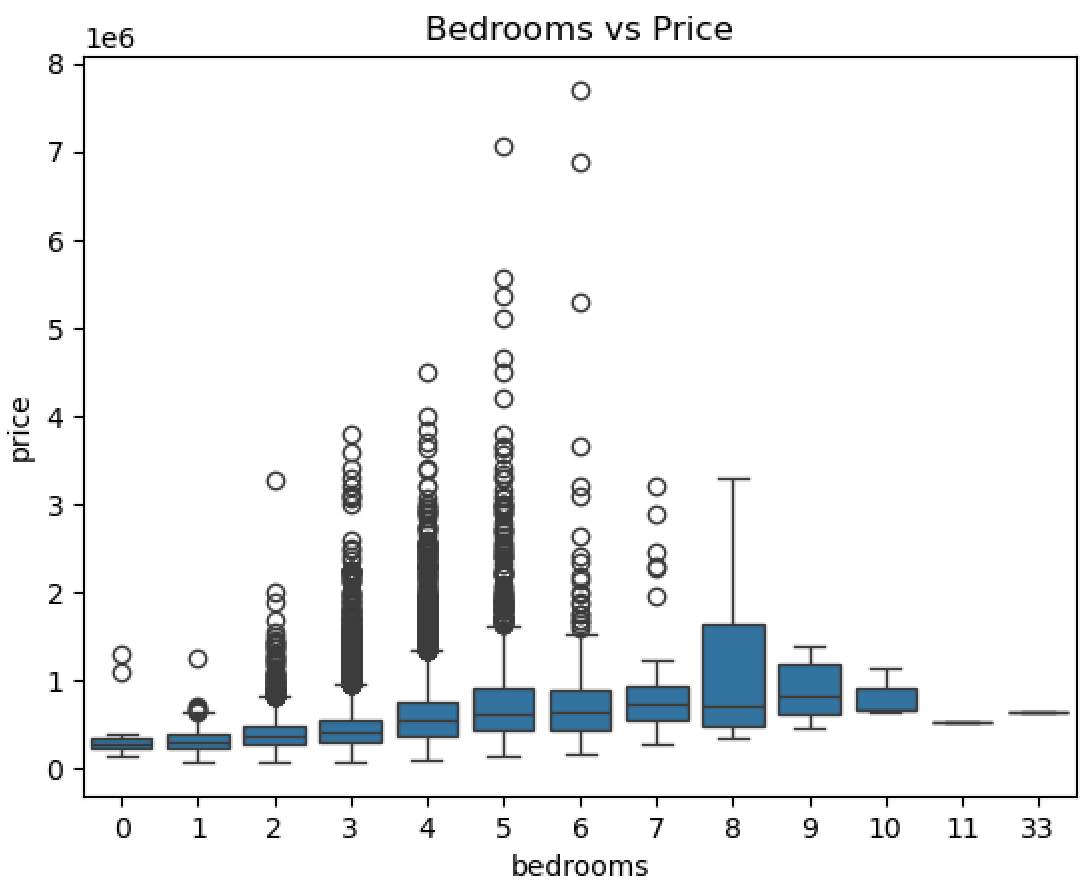
In [7]:
```python
df['house_age'] = 2025 - df['yr_built']
```

In [8]:
```python
df['yr_renovated'] = df['yr_renovated'].replace(0, np.nan)
```

In [9]:
```python
sns.histplot(df['price'], bins=50, kde=True)
plt.title('House Price Distribution')
plt.show()
```

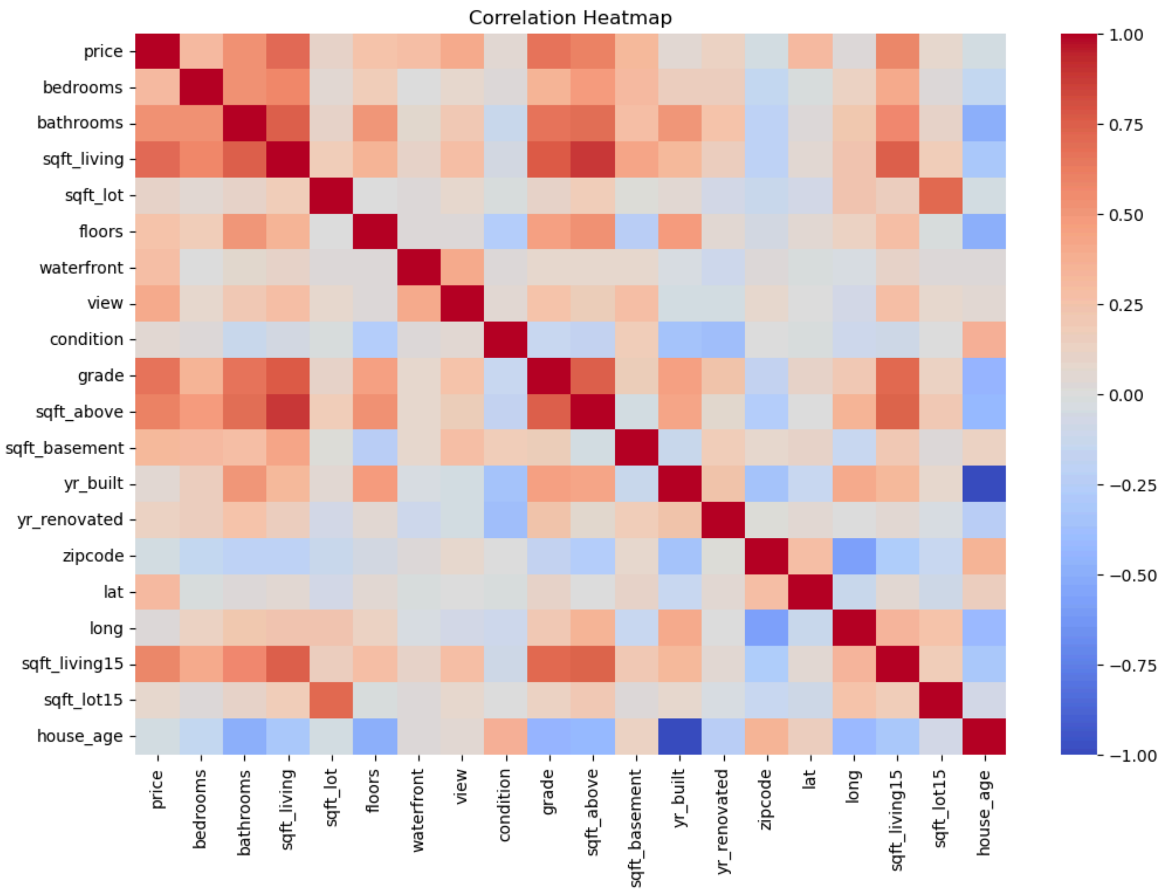


In [10]:
```
sns.boxplot(x='bedrooms', y='price', data=df)
plt.title('Bedrooms vs Price')
plt.show()
```



In [11]:
```
sns.scatterplot(x='sqft_living', y='price', data=df)
plt.title('Sqft Living vs Price')
plt.show()
```

In [12]:
```
plt.figure(figsize=(12,8))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



In [13]:
```python
X = df.drop('price', axis=1)
y = df['price']
```

In [14]:
```python
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [15]:
```python
# Fill yr_renovated NaN with 0 BEFORE splitting
df['yr_renovated'] = df['yr_renovated'].fillna(0)

# If you created house_age feature
df['house_age'] = df['house_age'].fillna(df['house_age'].median())
```

In [16]:
```python
# Features
X = df.drop('price', axis=1)

# Target
y = df['price']
```

In [17]:
```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [18]:
```python
X_train.isnull().sum()
X_test.isnull().sum()
```

Out[18]:
```
bedrooms         0
bathrooms        0
sqft_living      0
sqft_lot         0
floors           0
waterfront       0
view             0
condition        0
grade            0
sqft_above       0
sqft_basement    0
yr_built         0
yr_renovated     0
zipcode          0
lat              0
long             0
sqft_living15    0
sqft_lot15       0
house_age        0
dtype: int64
```

In [19]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

In [20]:
```
y_pred = model.predict(X_test)
```

In [21]:
```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)


mae, rmse, r2
```

Out[21]:
```
(127493.34208658694, 212539.5166381678, 0.7011904448878686)
```

In [ ]: